

PRACTICAL EXPERIMENT – 1

Name: K ANSHU

Roll No: 2520030440S

Section: 8

Title

Implementation of Process Creation and Command Execution in Linux using fork(), exec(), and wait() System Calls.

Aim

To develop a C program that accepts a Linux command from the user, creates a child process using fork(), executes the command, waits for the child process, and displays Parent and Child PIDs.

Theory

Process creation in Linux can be performed using the fork() system call. The fork() system call creates a new child process. The child process can execute a Linux command using an exec() family system call such as execlp(). The parent process can wait for the child process to complete using wait(). Process IDs (PIDs) are obtained using getpid() and the parent PID using getppid().

Algorithm

1. Start the program.
 2. Accept a Linux command from the user.
 3. Remove the newline character from the input.
 4. Create a child process using fork().
 5. If fork() fails, display an error message and terminate.
 6. In the child process, display the Child PID and Parent PID.
 7. Execute the entered Linux command using execlp().
 8. In the parent process, display the Parent PID.
 9. Wait for the child process to complete using wait().
 10. Display the child process completion message.
 11. Stop the program.
-

Code

Paste the **same code you successfully executed**:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <string.h>


int main()
{
    char command[100];

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if(pid < 0)
    {
        printf("Fork Failed!\n");
        return 1;
    }
    else if(pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execlp(command, command, NULL);

        printf("Command execution failed!\n");
    }
}
```

```
        exit(1);
    }
    else
    {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());

        wait(NULL);
        printf("Child process completed.\n");
    }

    return 0;
}
```

Execution Screenshots

Installation and successful compilation

```
k_anusri@DESKTOP-TM20BAK × + v
k_anusri@DESKTOP-TM20BAK:~$
k_anusri@DESKTOP-TM20BAK:~$ cat > exp1_process.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    char command[100];

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if(pid < 0)
    {
        printf("Fork Failed!\n");
        return 1;
    }
    else if(pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execlp(command, command, NULL);

        printf("Command execution failed!\n");
        exit(1);
    }
    else
    {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());

        wait(NULL);
        printf("Child process completed.\n");
    }

    return 0;
}
EOF
```

Program execution using lscpu

```
k_anusri@DESKTOP-TM20BAQ:~$ gcc expl_process.c -o expl_process && ./expl_process
Enter a Linux command: lscpu

Parent Process
Parent PID: 669

Child Process
Child PID : 670
Parent PID: 669
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:         39 bits physical, 48 bits virtual
Byte Order:            Little Endian
CPU(s):                16
On-line CPU(s) list:   0-15
Vendor ID:             GenuineIntel
Model name:            13th Gen Intel(R) Core(TM) i5-13450HX
CPU family:            6
Model:                 183
Thread(s) per core:    2
Core(s) per socket:    8
Socket(s):             1
Stepping:              1
BogoMIPS:              5222.40
Flags:                 fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr ss
e sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology tsc_reliable
nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pcid sse4_1 sse4_2 x2api
c movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowpre
fetch ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust bmi
1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv
1 xsave avx_vnni vnni umip waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_cl
ear serialize ibt flush_lld arch_capabilities

Virtualization features:
Virtualization:        VT-x
Hypervisor vendor:     Microsoft
Virtualization type:   full
Caches (sum of all):
L1d:                   384 KiB (8 instances)
L1i:                   256 KiB (8 instances)
L2:                    10 MiB (8 instances)
L3:                    20 MiB (1 instance)
NUMA:
NUMA node(s):          1
NUMA node0 CPU(s):    0-15
Vulnerabilities:
Gather data sampling:  Not affected
Ghostwrite:           Not affected
Indirect target selection: Not affected
Itlb multihit:        Not affected
L1tf:                 Not affected
Mds:                  Not affected
Meltdown:             Not affected
Mmio stale data:      Not affected
Old microcode:        Not affected
Reg file data sampling: Mitigation; Clear Register File
Retbleed:             Mitigation; Enhanced IBRS
Spec rstack overflow: Not affected
Spec store bypass:    Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1:           Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:           Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSE-eIBRS SW sequence; BHI BHI_D
IS_S
Srbds:                Not affected
Tsa:                  Not affected
Tsx async abort:      Not affected
Vmscape:              Not affected
Child process completed.
```

Program execution using uname

```
k_anusri@DESKTOP-TM20BAQ:~$ uname
Linux
```

Program execution using lsblk

```
k_anusri@DESKTOP-TM20BAQ:~$ lsblk
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda   8:0    0 356.9M  1 disk
sdb   8:16   0 159.4M  1 disk
sdc   8:32   0    2G    0 disk [SWAP]
sdd   8:48   0    1T    0 disk /mnt/wslg/distro
/
```

Program execution using ps

```
k_anusri@DESKTOP-TM20BAQ:~$ ps
  PID TTY          TIME CMD
  615 pts/0        00:00:00 bash
  752 pts/0        00:00:00 ps
```

Program execution using top

```
k_anusri@DESKTOP-TM20BAQ:~$ top
top - 06:02:13 up 15 min, 1 user, load average: 0.06, 0.04, 0.00
Tasks: 25 total, 1 running, 23 sleeping, 0 stopped, 1 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 7783.7 total, 7193.1 free, 567.7 used, 175.1 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 7215.9 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM    TIME+  COMMAND
    1 root        20   0   24120   15376 11584 S   0.0   0.2   0:00.38 systemd
    2 root        20   0    3180    2208   2072 S   0.0   0.0   0:00.00 init-systemd(Ub
    6 root        20   0    3180    2044    1940 S   0.0   0.0   0:00.00 init
   43 root        19  -1   50400   16764   15488 S   0.0   0.2   0:00.10 systemd-journal
   78 systemd+    20   0   22416   14448   11948 S   0.0   0.2   0:00.03 systemd-resolve
   84 root        20   0   35420   12368    9228 S   0.0   0.2   0:00.28 systemd-udev
  104 root        20   0    2888    1948    1820 S   0.0   0.0   0:00.03 chronyd-starter
  105 root        20   0    4472    3160    2908 S   0.0   0.0   0:00.00 cron
  106 message+   20   0    8812    5364    4684 S   0.0   0.1   0:00.03 dbus-daemon
  110 root        20   0   44096   29776   14632 S   0.0   0.4   0:00.16 networkd-dispat
  137 root        20   0   18436    9420    8180 S   0.0   0.1   0:00.06 systemd-logind
  181 syslog      20   0  220548    5728   4432 S   0.0   0.1   0:00.04 rsyslogd
  204 _chrony      20   0   23892   11348   7144 S   0.0   0.1   0:00.04 chronyd
  216 _chrony      20   0   12096    2452    1808 S   0.0   0.0   0:00.00 chronyd
  275 root        20   0    5492    2752    2568 S   0.0   0.0   0:00.00 agetty
  278 root        20   0  123460   32712   18092 S   0.0   0.4   0:00.10 unattended-upgr
  329 root        20   0      0      0      0 Z   0.0   0.0   0:00.00 SessionLeader
  331 root        20   0    8644    5560    4720 S   0.0   0.1   0:00.00 login
  374 k_anusri    20   0   22336   13532   10920 S   0.0   0.2   0:00.08 systemd
  376 k_anusri    20   0   22912    4088    2088 S   0.0   0.1   0:00.00 (sd-pam)
  400 k_anusri    20   0    6136    5472    3928 S   0.0   0.1   0:00.02 bash
  612 root        20   0    3184    1120     980 S   0.0   0.0   0:00.00 SessionLeader
  613 root        20   0    3200    1260    1108 S   0.0   0.0   0:00.01 Relay(615)
  615 k_anusri    20   0    6268    5596    3928 S   0.0   0.1   0:00.02 bash
  762 k_anusri    20   0    8312    6144    3888 R   0.0   0.1   0:00.15 top
```

Result

The program was executed successfully. Linux commands were executed through the child process and the parent process waited until completion.